

```
import keras

import numpy as np

import matplotlib.pyplot as plt


# Load Fashion MNIST dataset

fashion_mnist = keras.datasets.fashion_mnist


(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()


print(train_labels.shape)


# Class names
class_names = [
    'T-shirt/top',
    'Trouser',
    'Pullover',
    'Dress',
    'Coat',
    'Sandal',
    'Shirt',
    'Sneaker',
    'Bag',
    'Ankle boot'
]


# Training data shape
print(train_images.shape)


# Normalize pixel values (0 to 1)
train_images = train_images / 255.0
test_images = test_images / 255.0


# Reshape for CNN input
train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))
```

```
# Display sample images

plt.figure(figsize=(10, 10))

for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    plt.imshow(train_images[i].reshape(28, 28), cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])

plt.show()

# Build CNN model

model = keras.Sequential([
    keras.layers.Conv2D(
        32,
        kernel_size=(3, 3),
        activation='relu',
        input_shape=(28, 28, 1)
    ),

    keras.layers.MaxPooling2D(pool_size=(2, 2)),

    keras.layers.Flatten(),

    keras.layers.Dense(128, activation='relu'),

    keras.layers.Dense(10, activation='softmax')
])

# Model summary

model.summary()
```

```
# Compile model

model.compile(

    optimizer='adam',

    loss=keras.losses.SparseCategoricalCrossentropy(),

    metrics=['accuracy']

)
```

```
# Train model
```

```
model.fit(

    train_images,

    train_labels,

    epochs=5,

    verbose=1

)
```

```
# Evaluate model
```

```
test_loss, test_acc = model.evaluate(

    test_images,

    test_labels,

    verbose=2

)
```

```
print("\nTest accuracy =", test_acc * 100)
```

```
print("\nTest loss =", test_loss)
```

```
# Predictions
```

```
predictions = model.predict(test_images)
```

```
# Function to plot prediction
```

```
def plot_image(i, predictions_array, true_label, img):
```

```
    true_label, img = true_label[i], img[i]
```

```
    plt.grid(False)
```

```
plt.xticks([])
```

```
plt.yticks([])
```

```
plt.imshow(img.reshape(28, 28), cmap=plt.cm.binary)
```

```
predicted_label = np.argmax(predictions_array)
```

```
if predicted_label == true_label:
```

```
    color = 'blue'
```

```
else:
```

```
    color = 'red'
```

```
plt.xlabel(
```

```
    "{} {:.2f}% {}".format(
```

```
        class_names[predicted_label],
```

```
        100 * np.max(predictions_array),
```

```
        class_names[true_label]
```

```
    ),
```

```
    color=color
```

```
)
```

```
# Plot predictions
```

```
rows = 5
```

```
cols = 3
```

```
total_images = rows * cols
```

```
plt.figure(figsize=(10, 10))
```

```
for i in range(total_images):
```

```
    plt.subplot(rows, cols, i + 1)
```

```
    plot_image(i, predictions[i], test_labels, test_images)
```

```
plt.tight_layout()
```

```
plt.show()
```